

TEMA 1.

Conceptos

- **API** (Application Programming Interface): La interfaz de programación de aplicaciones es un conjunto de funciones y procedimientos cuyo uso permite a los desarrolladores interactuar con determinados elementos del sistema operativo o de otros programas.
- **APP**: Acrónimo de aplicación Informática.
- **Framework**: Entorno de trabajo formado por un conjunto de elementos o módulos de software que sirven como base para la organización y desarrollo de software.
- **Gadget**: Dispositivo tecnológico y novedoso, de pequeñas proporciones que tiene función específica y práctica.
- **Kernel**: Núcleo, parte central y fundamental del sistema operativo. Suele ser responsable de la comunicación entre el software y el hardware del sistema informático.
- **Runtime**: Intervalo de tiempo en el que un programa de (Computadora se ejecuta de la ejecución de un programa en un SO.
- **UI** (User Interface): Es la vista que permite el usuario interactuar de manera efectiva y cómoda con el sistema.
- **WAP** (Wireless Application Protocol): Estándar Internacional para APPs y dispositivos con comunicaciones inalámbricas.

Características dispositivos móviles:

- **Capacidad de Procesado**: Capacidad de cálculo y almacenamiento, necesario para el procesado y persistencia de la información.
- **Tamaño**: Portátiles, Resistentes, Gran Pantalla y Livianos.
- **Movilidad**: No dependencia del cableado. Batería duradera y comunicación inalámbrica.
- **Conectividad**: Conexión inalámbrica con gran alcance para la comunicación entre dispositivos.

Tipos de dispositivos móviles

- Limited Data Mobile Device: Pantalla pequeña, tipo texto con SMS y WAP.
- Basic Data Mobile Device: Pantalla mediana, menú navegación basado en interfaz gráfica, posibilidad de conexiones navegación web.
- Enhanced Data Mobile Device: Pantalla mediana Grande con interfaz gráfica, navegación web, Correo, GPS, cámara y sensores.

Generaciones Redes Moviles

Generación	Año	Características	Velocidad
1G	1980	Solo voz	2,4 KHz
2G	1990	Voz y SMS	64 KHz
3G	2001	Voz y datos, Multimedia	2 MB/s
4G	2010	Protocolo IP	100 MB/s
5G	2020	Banda Ancha	1 GB/s
6G	¿?	Satelital	¿?

Limitaciones:

- Principales: Capacidad de procesamiento, Almacenamiento, Manejo de elementos Multimedia.
- Recomendaciones: Evitar programar solo pensando en la gama tope. Optimizar el uso del procesador, Considerar las resoluciones de pantalla, tener en cuenta que no todos los dispositivos tienen los mismos sensores o tecnologías básicas.
- Aspectos a considerar: Las diferentes resoluciones pueden afectar a los resultados desde el desarrollo. Compatibilidad (errores) entre plataformas operativas y versiones. Cuidado al incorporar librerías y extensiones de terceros, no todos los equipos las admiten.

Sistemas Operativos Móviles

Capas:

1. Núcleo (Kernel): La base que gestiona el hardware, memoria y archivos.
2. Middleware: Módulos para aplicaciones, maneja mensajería y multimedia.

3. Entorno de ejecución: Permite a desarrolladores crear software.
4. Interfaces de usuario: La parte visual (botones, pantallas) que interactúa con el usuario.
5. Aplicaciones nativas: Software específico de cada fabricante.

Android:

Historia y Base:

- Lanzado en 2007 por Google y OHA (78 compañías)
- Sistema abierto con dos licencias: GNU GPLv2 (kernel) y APACHE v2 (resto)

Arquitectura por capas:

1. Kernel Linux: Base del sistema (ART)
2. HAL: Conecta hardware con software
3. Runtime: Ejecuta apps con archivos DEX
4. Bibliotecas C/C++: Código nativo
5. Framework Java API: Funciones del sistema
6. Apps del sistema: Aplicaciones básicas (correo, SMS, etc.)

IOS:

Origen:

- Desarrollado por Apple
- Derivado de Mac OS X
- Destaca por integración hardware-software y control táctil

Arquitectura (de arriba abajo):

1. Cocoa Touch: Capa para apps e interfaz (UIKit + Foundation)
2. Media Services: Audio y multimedia
3. Core Services: Servicios básicos (datos, red)
4. Core OS: Núcleo del sistema
5. iPhone Hardware: Base física

Windows Phone:

Datos básicos:

- Desarrollado por Microsoft (antes Windows Mobile)
- Lanzado en 2010
- Basado en Windows CE
- Interfaz Metro con mosaicos dinámicos

Arquitectura en 3 niveles:

1. Modelo de aplicación: Usa paquetes XAP
2. Modelo de UI: Gestiona interacciones
3. Integración en la nube: Conecta con servicios como:
 - Exchange
 - Google Mail
 - Hotmail

BlackBerry OS

- Desarrollado por RIM
- Enfocado en correo electrónico
- Requiere cuenta de desarrollador
- Basado en microkernel

Symbian

- Usado por Nokia, Sony Ericsson, Samsung
- Evolución del sistema Epoc
- Arquitectura microkernel
- Perdió relevancia con Android/iOS

Palm OS/WebOS

- Creado en 1996 por Palm
- Basado en Linux
- Interfaz web (HTML5, JavaScript, CSS)

Firefox OS

- Por Mozilla
- Basado en HTML5

- Usa JavaScript y Open Web APIs

Ubuntu Touch

- Por Canonical
- Basado en Linux
- Para smartphones y tablets

Harmony OS

- Por Huawei
- Distribuido y alta eficiencia
- Compatible con Android

Tipos Desarrollo para móviles

Desarrollo Nativo

- Específico para cada plataforma (iOS, Android)
- Ventajas: mejor rendimiento y control
- Desventaja: código específico para cada sistema

Desarrollo Multiplataforma compilado a nativo

- Compatible con varias plataformas
- Ejemplo: Xamarin (C# y .NET)
- Permite reutilizar código
- Requiere algunos ajustes específicos

Desarrollo Multiplataforma basado en HTML5

- Usa tecnologías web estándar
- Compatible con todas las plataformas
- Ejemplo: PhoneGap/Apache Cordova
- Menor rendimiento pero mayor compatibilidad

Entornos de desarrollo para diferentes plataformas móviles

Symbian

- Usa C++, Qt, Html, Python, Ruby, Java

- Herramienta: Qt IDE
- Incluye editor, compilador y emulador
- Es propietario pero tiene versiones demo

BlackBerry OS

- Usa BlackBerry Java Development Environment
- Desarrolla BlackBerry Java Applications
- Incluye APIs y MIB
- Herramientas optimizadas específicas

Windows Phone

- Soporta Silverlight (XAML)
- XNA para juegos (gráficos 2D/3D)
- Usa Visual Studio 2010
- SDK específico para Windows Phone

iOS

- Usa Swift o Objective-C
- Entorno: Xcode
- Incluye Interface Builder
- Herramientas de depuración integradas

Android

- Usa Java o Kotlin y XML.
- Herramienta: Android Studio
- Incluye editor, compilador y emulador

TEMA 2.

Glosario

- Aplicaciones responsivas: Se adaptan a diferentes tamaños y proporciones de pantalla, reorganizando o escalando contenido.
- Broadcast: En Android, transmisión de datos a todos los elementos activos del dispositivo.
- Bundle: Paquetes para pasar información entre actividades en Android.
- Gradle Scripts: Archivos en Android con información para compilar el proyecto.
- Intent: Elemento para cambiar entre pantallas o actividades en Android.
- SDK: Conjunto de herramientas para desarrollar aplicaciones en un sistema específico.
- Platform: Tecnologías base para desarrollar aplicaciones o implementar procesos.

Componentes de una aplicación

- Activity: Representa una pantalla en la app con la que interactúa el usuario.
- Service: Proceso en segundo plano para tareas sin interacción, como descargar archivos.
- Content Provider: Gestiona y comparte datos entre aplicaciones.
- Broadcast Receiver: Escucha eventos del sistema y reacciona, como cambios en batería o red.

Android Manifest

-El archivo Manifest de Android es esencial para el funcionamiento de las aplicaciones. El sistema lo usa para entender cómo ejecutar e interactuar con la app. Todas las apps deben tener un Manifest. Declara permisos, bibliotecas, requisitos de software/hardware y el nivel mínimo de API.

- Manifest: Define el espacio de nombres, paquete y atributos principales.
- Application: Incluye metadatos (título, ícono, tema) y componentes como actividades, servicios y receptores.
- Uses-SDK: Especifica las versiones y nivel mínimo del SDK.

Uses-Permission: Declara permisos necesarios para la app, como:

- INTERNET: Acceso a Internet.
- READ/WRITE_CONTACTS: Leer/escribir contactos.
- SEND_SMS: Enviar SMS.

- ACCESS_COARSE/FINE_LOCATION: Localización por telefonía/GPS.
- BLUETOOTH: Uso de Bluetooth.

- INTERNET: Conexión a Internet.
- READ_CONTACTS: Leer en la lista de contactos.
- WRITE_CONTACTS: Escribir en la lista de contactos.
- SEND_SMS: Enviar SMS.
- ACCESS_COARSE_LOCATION: Localización mediante telefonía.
- ACCESS_FINE_LOCATION: Localización mediante GPS.
- BLUETOOTH: Permite el uso del Bluetooth.
- CARPETA RES / XML: Almacena archivos XML.
- CARPETA RES / RAW: Se usa para almacenar los archivos raw de la aplicación (audio y video)

La clase R

La clase R contiene variables estáticas que identifican los recursos de la aplicación y los carga en memoria desde el archivo XML. Cada atributo tiene una dirección de memoria asociada a un recurso específico. A pesar de las mejoras en Android Studio, pueden ocurrir errores en la clase R, como enlaces rotos en archivos XML, lo que impide avanzar en el desarrollo. Para solucionar estos problemas, es importante verificar los archivos XML y usar herramientas como Build/Clean Project o Build/Rebuild Project.